IFCT0109 - MF0489_3 - Unidad Didactica 2

PKI

Cuaderno de Practicas Completo

OpenSSL + EJBCA Community

Kali Linux via Guacamole

Alumno/a: _____	**Fecha:** _____
Grupo: _____	**Calificacion:** _____

INFO Este cuaderno contiene 7 practicas progresivas sobre PKI. Las practicas 1-5 usan OpenSSL en Kali Linux. Las practicas 6-7 usan EJBCA Community como CA corporativa real. Accede a tu maquina en juanprofesor.com

Practicas de la unidad

N	Practica	Tiempo	Nivel
1	Explorar certificados de sitios web reales	30 min	Guiado
2	Crear una CA raiz con OpenSSL	45 min	Guiado
3	CA intermedia y cadena de confianza	45 min	Guiado
4	Emitir certificados y gestionar CSR	45 min	Autonomo
5	Revocacion de certificados y CRL	30 min	Guiado
6	EJBCA Community como CA corporativa	60 min	Autonomo
7	Certificado de servidor web con EJBCA	45 min	Autonomo

Practica 1 Explorar certificados de sitios web reales

Tiempo: 30 min
Guiado

Antes de crear nuestra propia PKI, vamos a explorar certificados reales de sitios web para entender su estructura y contenido.

1.1 Ver el certificado de un sitio web

```
# Conectar al servidor y ver el certificado completo
openssl s_client -connect www.google.com:443 -showcerts 2>/dev/null | \
  openssl x509 -text -noout | head -60

# Ver solo los campos más importantes
openssl s_client -connect www.google.com:443 2>/dev/null | \
  openssl x509 -noout -subject -issuer -dates -serial
```

1.2 Verificar la cadena de confianza

```
# Ver toda la cadena de certificados
openssl s_client -connect www.google.com:443 -showcerts 2>/dev/null | \
  grep -E 'subject|issuer'

# Verificar con otro sitio
openssl s_client -connect www.boe.es:443 -showcerts 2>/dev/null | \
  openssl x509 -noout -subject -issuer -dates
```

1.3 Analizar las extensiones X.509 v3

```
# Ver Subject Alternative Names (SAN)
openssl s_client -connect www.google.com:443 2>/dev/null | \
  openssl x509 -noout -ext subjectAltName

# Ver KeyUsage y ExtendedKeyUsage
openssl s_client -connect www.google.com:443 2>/dev/null | \
  openssl x509 -noout -ext keyUsage,extendedKeyUsage

# Ver puntos de distribucion CRL y OCSP
openssl s_client -connect www.google.com:443 2>/dev/null | \
  openssl x509 -noout -ext crlDistributionPoints,authorityInfoAccess
```

1.4 Verificar estado OCSP

```
# Obtener el certificado del servidor
openssl s_client -connect www.google.com:443 2>/dev/null | \
  openssl x509 > /tmp/google_cert.pem

# Obtener la URL del OCSP del certificado
openssl x509 -in /tmp/google_cert.pem -noout -ocsp_uri

# Consultar el estado OCSP
openssl ocsp -issuer /tmp/google_cert.pem \
  -cert /tmp/google_cert.pem \
  -url $(openssl x509 -in /tmp/google_cert.pem -noout -ocsp_uri) \
  -text 2>/dev/null | grep -E 'good|revoked|error'
```

1.5 Preguntas

1. Anota el emisor (Issuer) del certificado de www.google.com. ¿Cuantos niveles tiene la cadena de confianza?

Subject: Issuer (CA intermedia): Issuer de la CA intermedia (CA raiz):

2. ¿Que SANs (Subject Alternative Names) tiene el certificado? ¿Para que sirven?

SANs encontrados: Para que sirven:

3. ¿Cuál es la fecha de caducidad del certificado y cual es el periodo de validez total?

notBefore: notAfter: Periodo de validez (dias):

Practica 2 Crear una CA raiz con OpenSSL

Tiempo: 45 min
Guiado

Vamos a crear nuestra propia Autoridad de Certificacion (CA) raiz desde cero usando OpenSSL. Esta CA simulara una empresa real: Clinica Dental Mediterraneo.

2.1 Crear la estructura de directorios

```
# Crear estructura completa de la CA
mkdir -p ~/pki-lab/{ca-raiz,ca-intermedia,certificados,crl}
mkdir -p ~/pki-lab/ca-raiz/{certs,crl,newcerts,private,requests}
chmod 700 ~/pki-lab/ca-raiz/private

# Inicializar ficheros de control
echo 1000 > ~/pki-lab/ca-raiz/serial
touch ~/pki-lab/ca-raiz/index.txt
echo 1000 > ~/pki-lab/ca-raiz/crlnumber

cd ~/pki-lab
ls -la ca-raiz/
```

2.2 Crear la clave privada de la CA raiz

```
# Generar clave privada RSA 4096 bits (offline, maxima seguridad)
openssl genrsa -aes256 \
  -out ~/pki-lab/ca-raiz/private/ca-raiz.key 4096

# Verificar la clave
openssl rsa -in ~/pki-lab/ca-raiz/private/ca-raiz.key \
  -text -noout | head -5

# Proteger la clave
chmod 400 ~/pki-lab/ca-raiz/private/ca-raiz.key
```

NOTA La CA raiz en un entorno real nunca esta conectada a internet (air-gapped). Solo se usa para firmar CAs intermedias y permanece offline el resto del tiempo.

2.3 Crear el certificado autofirmado de la CA raiz

```
# Crear certificado autofirmado valido 10 anos
openssl req -new -x509 -days 3650 \
  -key ~/pki-lab/ca-raiz/private/ca-raiz.key \
  -out ~/pki-lab/ca-raiz/certs/ca-raiz.crt \
  -subj '/C=ES/ST=Malaga/L=Malaga/O=Clinica Dental Mediterraneo/OU=CA
Raiz/CN=Clinica Dental Mediterraneo Root CA'

# Verificar el certificado
openssl x509 -in ~/pki-lab/ca-raiz/certs/ca-raiz.crt \
  -text -noout | grep -E 'Subject:|Issuer:|Not Before:|Not After:|CA:TRUE'
```

2.4 Verificar que es una CA valida

```
# El Subject y el Issuer deben ser identicos (autofirmado)
```

```
openssl x509 -in ~/pki-lab/ca-raiz/certs/ca-raiz.crt -noout -subject -issuer

# Verificar que tiene la extension CA:TRUE
openssl x509 -in ~/pki-lab/ca-raiz/certs/ca-raiz.crt -noout -ext
basicConstraints

# Huella digital del certificado
openssl x509 -in ~/pki-lab/ca-raiz/certs/ca-raiz.crt -noout -fingerprint -
sha256
```

2.5 Preguntas

1. En el certificado de la CA raiz el Subject y el Issuer son identicos. ¿Por qué? ¿Que significa que sea autofirmado?

Tu respuesta:

2. Anota la huella digital SHA-256 de tu CA raiz. ¿Para que se usa esta huella en la práctica?

SHA-256 fingerprint: Uso practico:

Practica 3 CA intermedia y cadena de confianza

Tiempo: 45 min
Guiado

En una PKI real la CA raiz nunca emite certificados de usuario directamente. Se usa una CA intermedia para aislar la CA raiz. Si la CA intermedia se compromete, la CA raiz sigue siendo segura.

3.1 Crear estructura de la CA intermedia

```
mkdir -p ~/pki-lab/ca-intermedia/{certs,crl,newcerts,private,requests}
chmod 700 ~/pki-lab/ca-intermedia/private
echo 2000 > ~/pki-lab/ca-intermedia/serial
touch ~/pki-lab/ca-intermedia/index.txt
echo 2000 > ~/pki-lab/ca-intermedia/crlnumber
```

3.2 Generar clave y CSR de la CA intermedia

```
# Clave privada de la CA intermedia (2048 bits, menos que la raiz)
openssl genrsa -aes256 \
  -out ~/pki-lab/ca-intermedia/private/ca-intermedia.key 2048

# Crear CSR de la CA intermedia
openssl req -new \
  -key ~/pki-lab/ca-intermedia/private/ca-intermedia.key \
  -out ~/pki-lab/ca-intermedia/requests/ca-intermedia.csr \
  -subj '/C=ES/ST=Malaga/L=Malaga/O=Clinica Dental Mediterraneo/OU=CA
Servidores/CN=Clinica Dental Mediterraneo Servers CA'

# Verificar el CSR
openssl req -in ~/pki-lab/ca-intermedia/requests/ca-intermedia.csr -text -
noout | head -20
```

3.3 Firmar la CA intermedia con la CA raiz

```
# La CA RAIZ firma el certificado de la CA INTERMEDIA
# Esto establece la cadena de confianza
openssl x509 -req -days 1825 \
  -in ~/pki-lab/ca-intermedia/requests/ca-intermedia.csr \
  -CA ~/pki-lab/ca-raiz/certs/ca-raiz.crt \
  -CAkey ~/pki-lab/ca-raiz/private/ca-raiz.key \
  -CAcreateserial \
  -out ~/pki-lab/ca-intermedia/certs/ca-intermedia.crt \
  -extensions v3_ca \
  -extfile <(echo
'[v3_ca]\nsubjectKeyIdentifier=hash\nauthorityKeyIdentifier=keyid:always,issuer\nba
sicConstraints=critical,CA:true,pathlen:0\nkeyUsage=critical,digitalSignature,cRLSi
gn,keyCertSign')

# Verificar quien firmo la CA intermedia
openssl x509 -in ~/pki-lab/ca-intermedia/certs/ca-intermedia.crt -noout -subject -
issuer
```

3.4 Crear y verificar la cadena completa

```
# Crear el fichero de cadena completa
cat ~/pki-lab/ca-intermedia/certs/ca-intermedia.crt \
    ~/pki-lab/ca-raiz/certs/ca-raiz.crt \
    > ~/pki-lab/ca-intermedia/certs/cadena-completa.crt

# Verificar la cadena de confianza
openssl verify \
  -CAfile ~/pki-lab/ca-raiz/certs/ca-raiz.crt \
  ~/pki-lab/ca-intermedia/certs/ca-intermedia.crt

# Ver la cadena completa
openssl crl2pkcs7 -nocrl \
  -certfile ~/pki-lab/ca-intermedia/certs/cadena-completa.crt | \
  openssl pkcs7 -print_certs -noout
```

3.5 Preguntas

1. Explica por qué pathlen:0 en la CA intermedia es importante para la seguridad de la PKI:

Tu respuesta:

2. ¿Si la CA intermedia se ve comprometida, que pasos habria que seguir y como afecta a los certificados ya emitidos?

Tu respuesta:

Practica 4 Emitir certificados y gestionar CSR

Tiempo: 45 min
Autonomo

En esta práctica cada alumno actua como usuario final que solicita un certificado a la CA de la Clinica Dental Mediterraneo. El instructor actua como CA y firma los CSR de los alumnos.

ESCENARIO	Eres el técnico de TI de la Clínica. Necesitas un certificado para tu servidor web interno (intranet.clinica-mediterraneo.es). Genera tu propio CSR y envíaselo al administrador de la CA (el instructor).

4.1 Generar tu par de claves

```
mkdir -p ~/pki-lab/mi-certificado
cd ~/pki-lab/mi-certificado

# Generar clave privada (sin contrasena para el servidor web)
openssl genrsa -out mi-servidor.key 2048

# Verificar la clave
openssl rsa -in mi-servidor.key -text -noout | head -3
```

4.2 Crear el CSR

```
# Crear CSR con Subject Alternative Names
openssl req -new \
  -key mi-servidor.key \
  -out mi-servidor.csr \
  -subj '/C=ES/ST=Malaga/L=Malaga/O=Clinica Dental
Mediterraneo/CN=intranet.clinica-mediterraneo.es' \
  -addext 'subjectAltName=DNS:intranet.clinica-
mediterraneo.es,DNS:intranet,IP:192.168.99.101'

# Verificar el contenido del CSR
openssl req -in mi-servidor.csr -text -noout

# Ver el CSR en formato PEM (para enviarlo al instructor)
cat mi-servidor.csr
```

4.3 El instructor firma el CSR

El instructor recibe tu CSR y lo firma con la CA intermedia. Recibes tu certificado firmado.

```
# (El instructor ejecuta esto con tu CSR)
# openssl x509 -req -days 365 \
#   -in alumno-XX.csr \
#   -CA ~/pki-lab/ca-intermedia/certs/ca-intermedia.crt \
#   -CAkey ~/pki-lab/ca-intermedia/private/ca-intermedia.key \
#   -CAcreateserial \
#   -out alumno-XX.crt \
#   -copy_extensions copy

# Tu recibes el fichero mi-servidor.crt del instructor
# Verificar tu certificado
openssl x509 -in mi-servidor.crt -text -noout
```

4.4 Verificar el certificado emitido

```
# Verificar contra la cadena de la CA
openssl verify \
  -CAfile ~/pki-lab/ca-intermedia/certs/cadena-completa.crt \
  mi-servidor.crt

# Verificar que la clave privada corresponde al certificado
openssl x509 -noout -modulus -in mi-servidor.crt | md5sum
openssl rsa -noout -modulus -in mi-servidor.key | md5sum
# Ambos MD5 deben ser identicos

# Ver numero de serie del certificado
openssl x509 -in mi-servidor.crt -noout -serial
```

4.5 Preguntas

1. Anota el número de serie de tu certificado. ¿Porque cada certificado tiene un número de serie único?

Número de serie: Porque es único:

2. ¿Qué ocurre si el MD5 de la clave privada y el certificado no coinciden? ¿En que situación podría pasar esto?

Tu respuesta:

Practica 5 Revocación de certificados y CRL

Tiempo: 30 min
Guiado

Simulamos que la clave privada de un certificado ha sido comprometida. El administrador de la CA debe revocar el certificado y publicar una nueva CRL.

5.1 Revocar un certificado

```
cd ~/pki-lab

# Revocar el certificado del alumno (motivo: keyCompromise)
openssl ca \
  -keyfile ca-intermedia/private/ca-intermedia.key \
  -cert ca-intermedia/certs/ca-intermedia.crt \
  -config <(cat << 'CONF'
[ ca ]
default_ca = CA_default
[ CA_default ]
database = /root/pki-lab/ca-intermedia/index.txt
CONF
) \
  -revoke mi-certificado/mi-servidor.crt \
  -crl_reason keyCompromise
```

5.2 Generar la CRL

```
# Generar nueva CRL firmada por la CA intermedia
openssl ca \
  -keyfile ~/pki-lab/ca-intermedia/private/ca-intermedia.key \
  -cert ~/pki-lab/ca-intermedia/certs/ca-intermedia.crt \
  -gencrl \
  -crldays 30 \
  -out ~/pki-lab/crl/ca-intermedia.crl

# Ver el contenido de la CRL
openssl crl -in ~/pki-lab/crl/ca-intermedia.crl -text -noout
```

5.3 Verificar que el certificado esta revocado

```
# Verificar el certificado contra la CRL
openssl verify \
  -CAfile ~/pki-lab/ca-intermedia/certs/cadena-completa.crt \
  -CRLfile ~/pki-lab/crl/ca-intermedia.crl \
  -crl_check \
  ~/pki-lab/mi-certificado/mi-servidor.crt

# Debe mostrar: error 23 at 0 depth lookup: certificate revoked
```

5.4 Preguntas

1. Que razones de revocación (crl_reason) existen en el estandar X.509? ¿Cuándo usarías cada una?

Razones de revocacion y cuando usarlas:

2. Si un atacante modifica la CRL para eliminar un certificado revocado, ¿qué mecanismo impide que esa CRL sea aceptada?

Tu respuesta:

BLOQUE EJBCA COMMUNITY — CA Corporativa Real

Las practicas 6 y 7 usan EJBCA Community, la plataforma PKI open source usada por empresas reales. El servidor EJBCA está en ejbca.juanprofesor.com y es accesible desde tu navegador en Kali.

Practica 6 EJBCA Community como CA corporativa

Tiempo: 60 min
Autonomo

EJBCA es la PKI open source más completa del mercado. En esta práctica interactuamos con ella como lo haría un técnico en una empresa real: vía REST API desde la línea de comandos.

EJBCA	Servidor PKI: ejbca.juanprofesor.com | Puerto REST: 443 | El administrador ha creado un perfil de certificado 'AlumnosLab' y una entidad final para cada alumno.

6.1 Explorar la interfaz web de EJBCA

Abre Firefox en Kali y navega a:

```
# En el navegador Firefox de tu Kali
https://ejbca.juanprofesor.com/ejbca/publicweb/

# Ver informacion publica de la CA
# -> CA Certificates and CRL -> descargar el certificado de la CA raiz
# -> Certificate Search -> buscar certificados emitidos
```

6.2 Configurar acceso a la API REST

```
cd ~/pki-lab

# Descargar el certificado raiz de EJBCA
curl -sk
https://ejbca.juanprofesor.com/ejbca/publicweb/webdist/certdist?cmd=cachain\&caid=-
1\&format=pem \
  -o ejbca-ca-chain.pem

# Verificar que se ha descargado correctamente
openssl x509 -in ejbca-ca-chain.pem -noout -subject -issuer 2>/dev/null || \
openssl storeutl -noout -text -certs ejbca-ca-chain.pem 2>/dev/null | grep -E
'Subject|Issuer' | head -6

# Probar la API REST
curl -sk --cacert ejbca-ca-chain.pem \
  https://ejbca.juanprofesor.com/ejbca/ejbca-rest-api/v1/certificate/status | \
  python3 -m json.tool
```

6.3 Generar CSR y solicitar certificado via REST API

```
# Generar clave y CSR para el certificado de alumno
# Sustituye ALUMNOXX por tu numero (alumno01, alumno02...)
ALUMNO='alumno01'

openssl genrsa -out ${ALUMNO}.key 2048
```

```
openssl req -new \
  -key ${ALUMNO}.key \
  -out ${ALUMNO}.csr \
  -subj "/C=ES/O=Clinica Dental Mediterraneo/CN=${ALUMNO}@clinica-
mediterraneo.es"

# Convertir CSR a Base64 para la API
CSR_B64=$(openssl req -in ${ALUMNO}.csr -outform DER | base64 -w0)

# Solicitar certificado via REST API de EJBCA
curl -sk --cacert ejbca-ca-chain.pem \
  -X POST \
  -H 'Content-Type: application/json' \
  -d "{\"certificate_request\": \"${CSR_B64}\", \"certificate_profile_name\":
\"ENDUSER\", \"end_entity_profile_name\": \"EMPTY\",
\"certificate_authority_name\": \"ManagementCA\", \"username\": \"${ALUMNO}\",
\"password\": \"foo123\"}" \
  https://ejbca.juanprofesor.com/ejbca/ejbca-rest-
api/v1/certificate/pkcs10enroll \
  -o respuesta.json

# Ver la respuesta
cat respuesta.json | python3 -m json.tool | head -20
```

6.4 Extraer y verificar el certificado

```
# Extraer el certificado de la respuesta JSON
cat respuesta.json | python3 -c \
  "import sys,json,base64; d=json.load(sys.stdin);
open('${ALUMNO}.crt','wb').write(base64.b64decode(d['certificate']))"

# Verificar el certificado recibido
openssl x509 -in ${ALUMNO}.crt -noout -subject -issuer -dates -serial

# Verificar la cadena de confianza
openssl verify -CAfile ejbca-ca-chain.pem ${ALUMNO}.crt

# Ver todos los campos
openssl x509 -in ${ALUMNO}.crt -text -noout
```

6.5 Preguntas

1. ¿Qué ventajas tiene usar una REST API para solicitar certificados frente al proceso manual con OpenSSL?

Tu respuesta:

2. ¿Qué diferencia hay entre el proceso de emisión en tu CA con OpenSSL (practica 4) y el de EJBCA? ¿Cual es más adecuado para una empresa con 500 empleados?

Tu respuesta:

Practica 7 Certificado de servidor web con EJBCA y nginx

Tiempo: 45 min
Autonomo

Simulamos el proceso completo de despliegue de un servidor web HTTPS corporativo: emitir el certificado desde EJBCA e instalarlo en un servidor nginx ejecutado en Docker en tu propio clon Kali.

7.1 Crear el servidor nginx en Docker

```
mkdir -p ~/servidor-web/{certs,html,nginx}
cd ~/servidor-web

# Pagina web de prueba
echo '<h1>Clinica Dental Mediterraneo - Intranet</h1>' > html/index.html
echo '<p>Servidor seguro con certificado EJBCA</p>' >> html/index.html

# Configuracion nginx con HTTPS
cat > nginx/nginx.conf << 'EOF'
server {
    listen 443 ssl;
    server_name intranet.clinica-mediterraneo.es;
    ssl_certificate /etc/nginx/certs/servidor.crt;
    ssl_certificate_key /etc/nginx/certs/servidor.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    location / { root /usr/share/nginx/html; }
}
EOF
```

7.2 Emitir certificado de servidor desde EJBCA

```
cd ~/servidor-web

# Generar clave privada del servidor
openssl genrsa -out certs/servidor.key 2048

# Crear CSR con SAN para el servidor web
openssl req -new \
  -key certs/servidor.key \
  -out certs/servidor.csr \
  -subj '/C=ES/O=Clinica Dental Mediterraneo/CN=intranet.clinica-
mediterraneo.es' \
  -addext 'subjectAltName=DNS:intranet.clinica-
mediterraneo.es,IP:192.168.99.101'

# Solicitar certificado a EJBCA
CSR_B64=$(openssl req -in certs/servidor.csr -outform DER | base64 -w0)

curl -sk --cacert ~/pki-lab/ejbca-ca-chain.pem \
  -X POST -H 'Content-Type: application/json' \
  -d "{\"certificate_request\": \"${CSR_B64}\", \"certificate_profile_name\":
\"ENDUSER\", \"end_entity_profile_name\": \"EMPTY\",
\"certificate_authority_name\": \"ManagementCA\", \"username\": \"servidor-
web\", \"password\": \"foo123\"}" \
  https://ejbca.juanprofesor.com/ejbca/ejbca-rest-
api/v1/certificate/pkcs10enroll \
```

```
  | python3 -c "import sys,json,base64; d=json.load(sys.stdin);
open('certs/servidor.crt','wb').write(base64.b64decode(d['certificate']))"

# Verificar el certificado
openssl x509 -in certs/servidor.crt -noout -subject -dates
```

7.3 Arrancar el servidor HTTPS

```
# Crear Docker Compose
cat > docker-compose.yml << 'EOF'
services:
  nginx:
    image: nginx:alpine
    ports:
      - '8443:443'
    volumes:
      - ./html:/usr/share/nginx/html
      - ./certs:/etc/nginx/certs
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf
EOF

# Arrancar nginx
docker compose up -d

# Verificar que escucha
docker compose ps
```

7.4 Verificar el servidor HTTPS

```
# Verificar con curl usando la CA de EJBCA
curl -v --cacert ~/pki-lab/ejbca-ca-chain.pem \
  https://localhost:8443 2>&1 | grep -E 'SSL|certificate|issuer|subject|HTTP'

# Ver el certificado que presenta el servidor
openssl s_client -connect localhost:8443 \
  -CAfile ~/pki-lab/ejbca-ca-chain.pem 2>/dev/null | \
  openssl x509 -noout -subject -issuer -dates

# En el navegador Firefox de Kali:
# Importar la CA de EJBCA en Firefox primero:
# Preferencias -> Privacidad -> Ver certificados -> Autoridades -> Importar
# Seleccionar ejbca-ca-chain.pem
# Luego navegar a: https://localhost:8443
```

7.5 Preguntas finales

1. Describe el proceso completo que ha seguido el certificado desde su solicitud hasta estar instalado en el servidor nginx:

Proceso completo (CSR -> EJBCA -> nginx -> HTTPS):

2. Un usuario que visite tu servidor sin tener la CA de EJBCA instalada en su navegador vería un aviso de seguridad. Explica por qué y cómo se solucionaría en un entorno empresarial real:

Tu respuesta:

Criterios de evaluacion

Practica	Nivel	Indicador de logro	Peso
P1 - Explorar certificados reales	Guiado	Analiza correctamente la cadena de confianza y los campos X.509	10%
P2 - CA raiz con OpenSSL	Guiado	Crea la CA raiz con estructura correcta y certificado autofirmado valido	15%
P3 - CA intermedia	Guiado	Firma correctamente la CA intermedia y verifica la cadena de confianza	15%
P4 - Emitir certificados CSR	Autonomo	Genera CSR valido, recibe y verifica el certificado firmado	15%
P5 - Revocacion y CRL	Guiado	Revoca correctamente y verifica que el certificado aparece en la CRL	10%
P6 - EJBCA REST API	Autonomo	Obtiene certificado valido de EJBCA via REST API y verifica la cadena	20%
P7 - Servidor web HTTPS	Autonomo	Despliega nginx con HTTPS funcional usando certificado de EJBCA	15%

ENTREGA	Entrega este cuaderno completado con todas las respuestas. Adjunta tambien un fichero .zip con los archivos generados en las practicas: ca-raiz.crt, ca-intermedia.crt, mi-servidor.crt, ejbca-alumnoXX.crt y capturas de pantalla del servidor nginx funcionando con HTTPS.